

01-波形发生器设计

由 Floyd-Fish

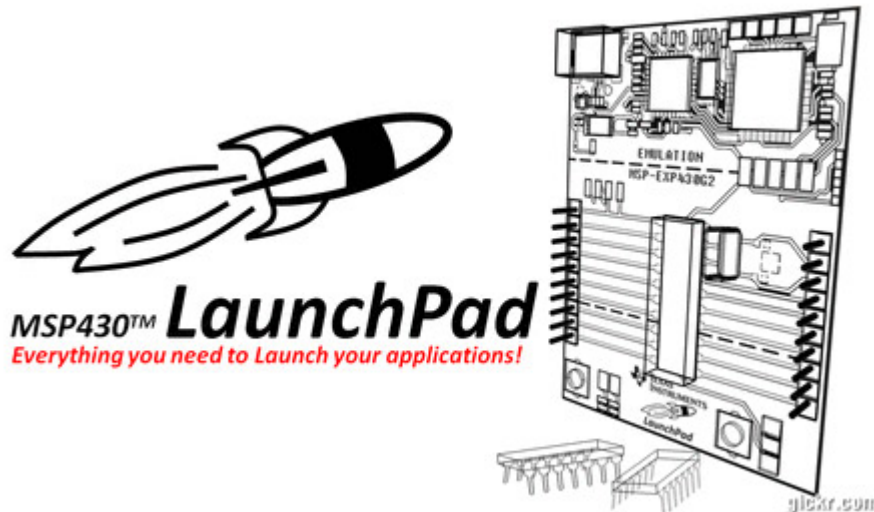
ID:1330

<https://www.emoe.xyz/project-waveform-generator/>

September 14, 2020

分类目录: 嵌入式, 所有文章, 硬件, 精密模拟, 综合项目

标签: MSP430, 项目



本文目录

波形发生器设计

前言

放题目,咬人

在New Thread里分析

Simulation

NE555搭建的锯齿波发生器

低通滤波器-10KHz

低通滤波器-30KHz

Coding时间到

PWM Module

Timer Module

我很认真.jpg

实物

代码?

下一期

波形发生器设计

前言

一直搞理论是不是太难懂了= =

学习电子当然是需要理论+实践相结合的方式咯~

总之这一个全新的系列，专门讲我们做过的各种(阴间)题目

我会共享我(们)的方案和设计思路，当然我(们)的方案不是最佳方案，如果你有更好的方案可以在评论区讨论哦！

放题目,咬人

使用题目指定的NE555芯片、一片通用四运放LM324芯片以及单片机，设计制作一个频率可变的同时输出脉冲波、锯齿波、正弦波I、正弦波II的波形产生电路，并采用单片机测量显示相关参数。

详细要求如下：

- 基本部分

1.同时四通道输出、每通道输出脉冲波、锯齿波、正弦波I、正弦波II中的一种波形，每通道输出的负载电阻均为600 欧姆。

2.四种波形的频率关系为1: 1: 1: 3 (3 次谐波)：脉冲波、锯齿波、正弦波I输出频率范围为 8kHz—10kHz，输出电压幅度峰峰值为1V；正弦波II输出频率范围为 24kHz—30kHz，输出电压幅度峰峰值为6V；脉冲波、锯齿波和正弦波输出波形应无明显失真（使用示波器测量时）。频率误差不大于10%；通带内输出电压幅度峰峰值误差不大于5%。

3.电源基本要求为 $\pm 10V$ ，要求预留脉冲波、锯齿波、正弦波I、正弦波II和电源的测试端子。每通道输出的负载电阻600 欧姆应标示清楚、置于明显位置，便于测试。

- 发挥部分

1.只选用+10V 单电源供电。脉冲波占空比程控可调，占空比调整在5%-95%之间。脉冲波频率程控可调，范围8KHz-10KHz。

2.搭建电路采用单片机测量脉冲波频率，正半周脉宽并测量显示频率、占空比。预留出单独可测频率脉宽端口（可以前面波形发生电路断开，单独测试）。

3.单片机按键步进控制输出直流电平0~3V，步进0.1V。（提示：可采用PWM+滤波方式产生）。

- Notes:

在New Thread里分析

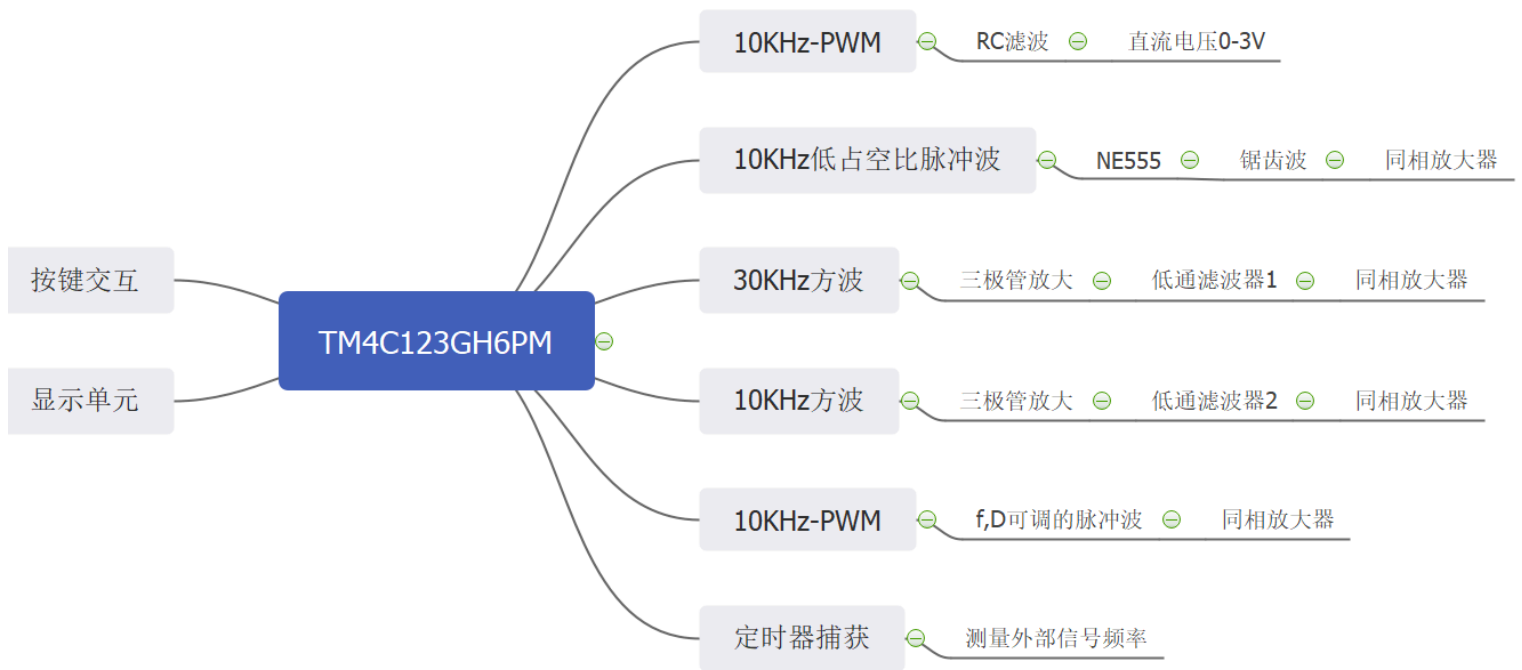
首先，题目要求只能用一片NE555和LM324，所以纯用运放+555来搭建方波、锯齿波、正弦波信号发生电路，然后还要让其带载稳幅是不太好实现的...

那么我们就想到了另一种思路：

1. 用单片机产生脉冲波、方波信号(通过定时器/PWM模块，这是非常容易实现的)；
2. 然后将占空比50%的方波通过低通滤波器(Low-Pass-Filter, LPF)(可以有源，可以无源，但无源LPF的幅频特性显示，其通带增益较平滑，在一定频率范围内变化较大,有源滤波器更加适合)；
3. 通过上一步的操作,我们可以滤出方波的基波正弦信号,也就是其同频率的正弦信号,再过一个运放跟随(或者同相放大器)来对其输出幅度进行调节,就可以实现题目要求的1Vpp输出要求。
4. 至于锯齿波(注意不是三角波!!!),可以利用电容近似线性的部分充电曲线和NE555的放电端，给2、6脚加上占空比极低的脉冲波，在电容端就可以得到较为理想的锯齿波。
5. 直流电压调节,题目提示也说得很明白了,直接PWM+RC低通滤波器硬怼就完事了~比如我单片机IO输出电压高电平是3.3V,低电平是0V,那么我输出一个10KHz的脉冲波信号,占空比50%,再通过一个截止频率100HZ以下的RC低通滤波器,得到的直流电压就是 $3.3/2 = 1.65V$ 。如果把占空比设置成20%,直流电压就是 $3.3 * 0.2 = 0.66V$ 。

单片机,我想着试试新鲜玩意儿,stm32用熟了,用用ti的tm4c123。(事实证明这单片机的软件开发非常坑爹。)

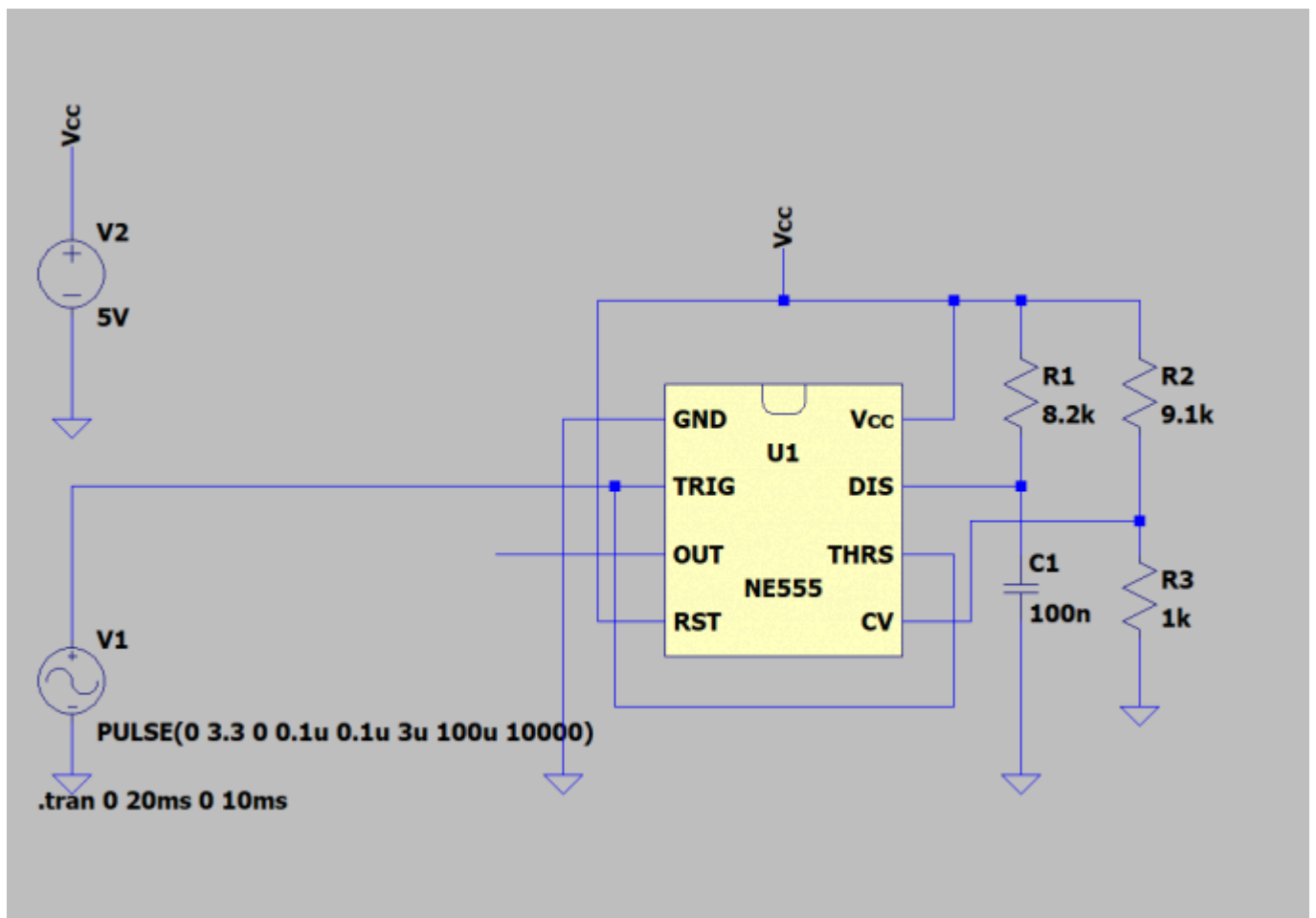
总的方案就是这样：



Simulation

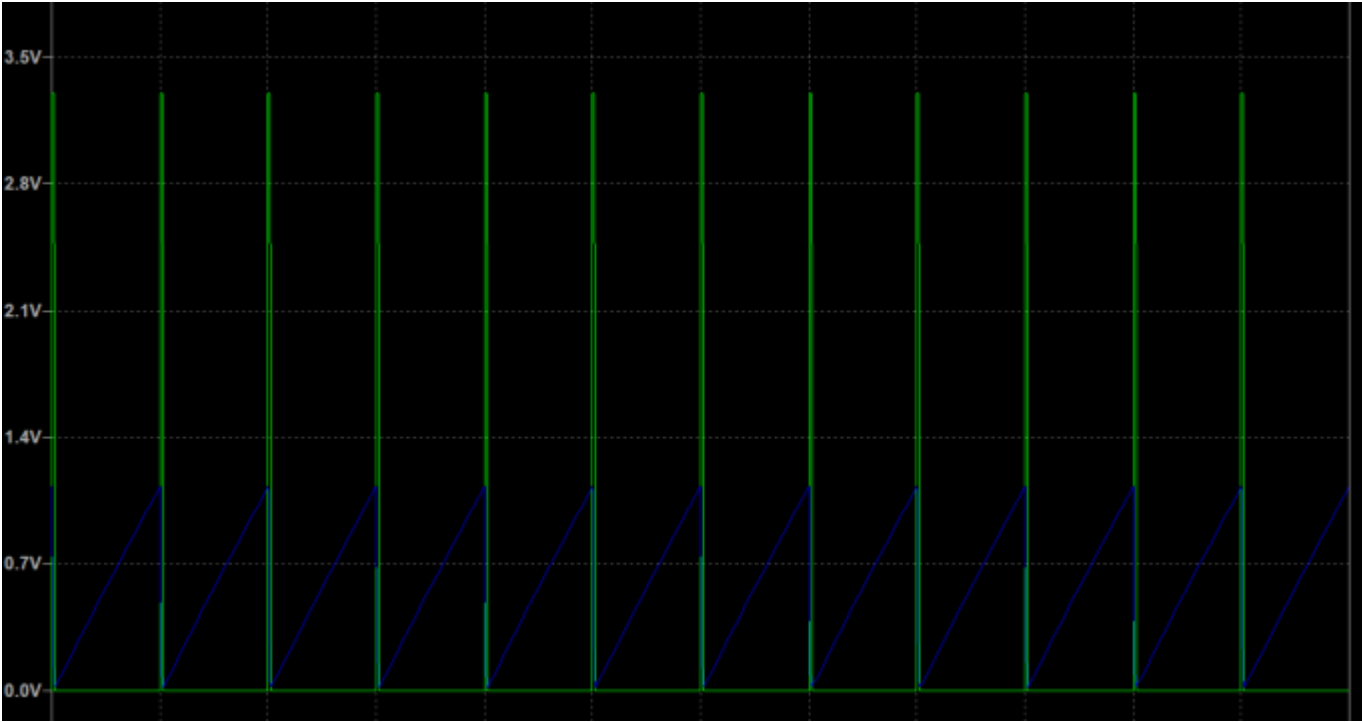
仿真这块,我使用了 **LTspice** ,软件很小巧,功能却不少~

NE555搭建的锯齿波发生器

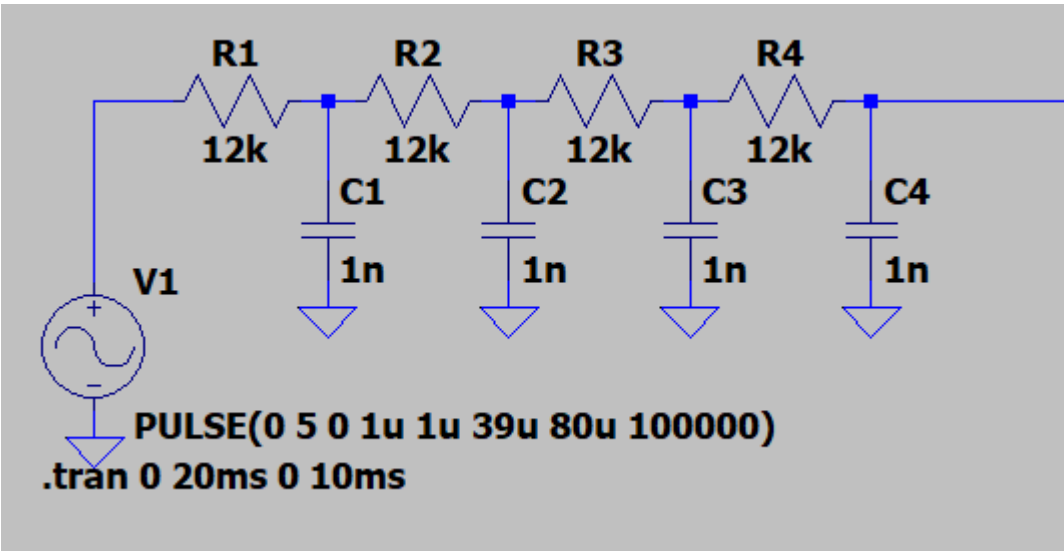


在2、6脚加上占空比极小($\leq 5\%$)的脉冲波,在脉冲波的低电平期间,电源通过R1给电容C1充电,电容的电压曲线近似线性;
在脉冲波的高电平期间,NE555的7脚会为电容放电,形成非常陡峭的下降沿,在电容C1上接一同相放大器(高输入阻抗,不影响NE555充放电回路),即可将锯齿波波形放大至所需要的1Vpp锯齿波信号(带载调试)

以下是仿真波形:

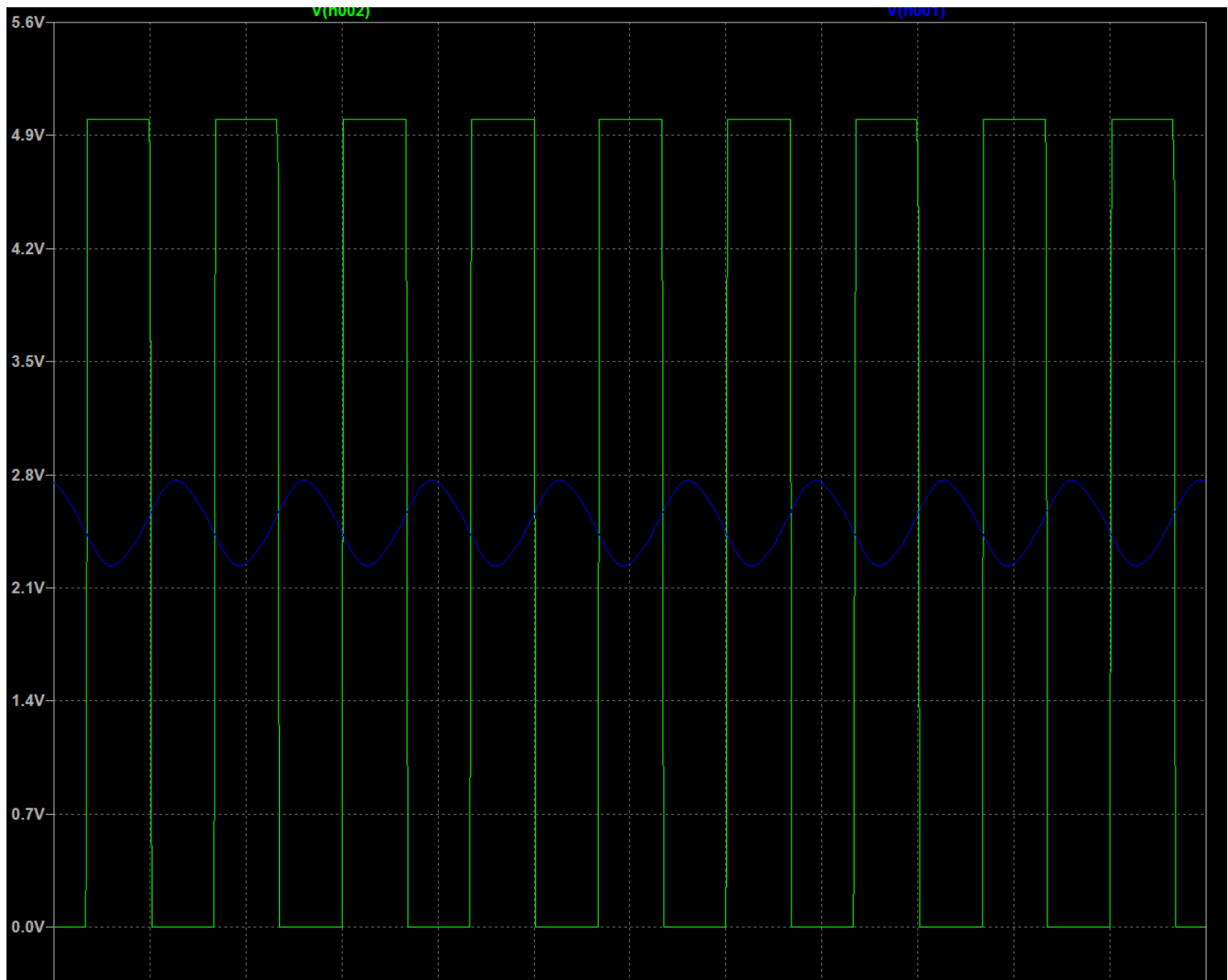


低通滤波器-10KHz



这里我图省事,直接用了4阶RC低通滤波器,从单片机的管脚输出10KHz的方波,经过三极管放大至电源轨电压(实际达不到,略低于,实测大约是电源电压-1v左右)
然后将放大过后的电压输入LPF,再输入同相放大器放大(缩小),即可得到1Vpp的10KHz正弦信号(带载调试)

以下是仿真波形:



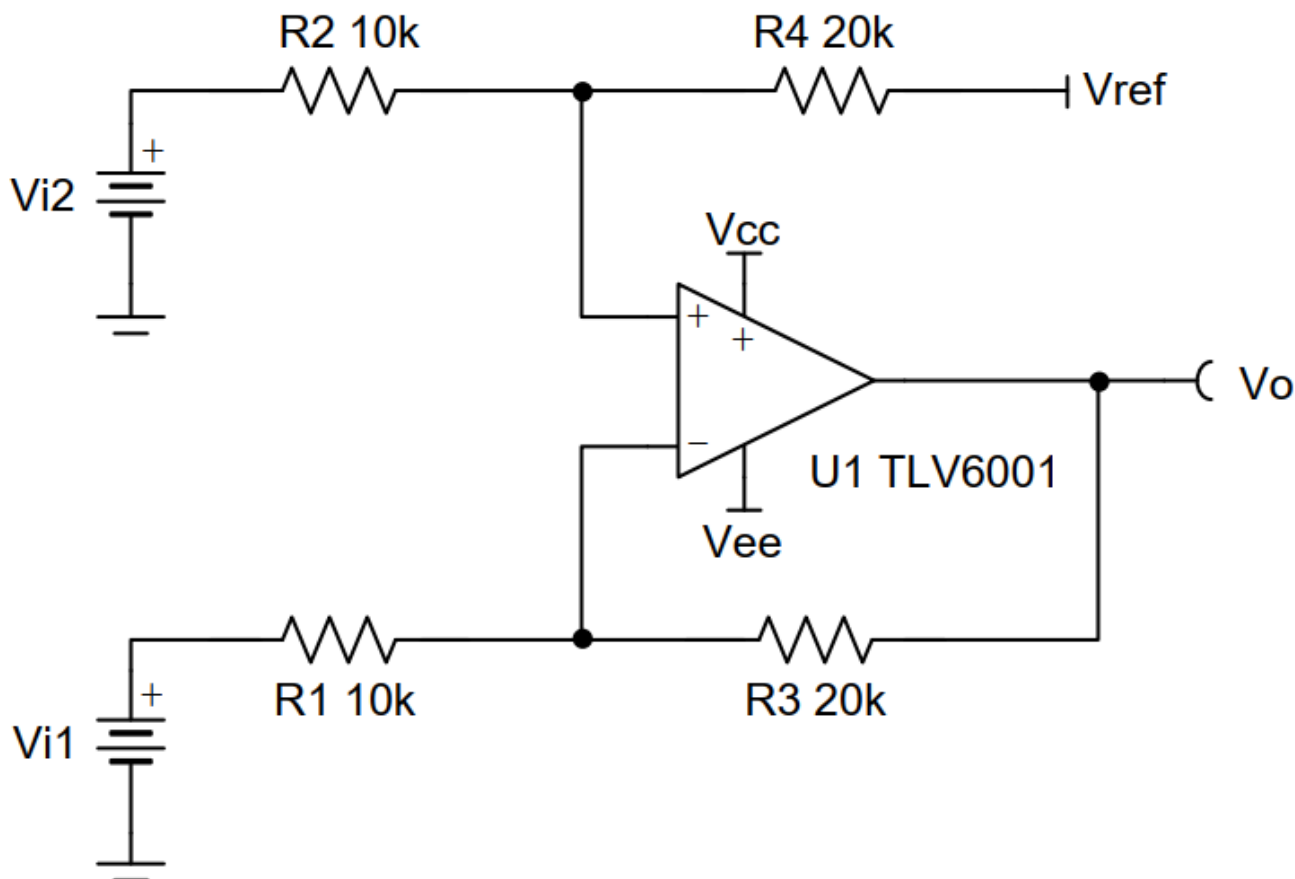
我们注意到,滤出来的正弦波带有较高的直流分量,如果放大很多倍的话可能会放飞(超出运放工作电压范围,输出保持在电源电压附近不变),我们可以使用差分放大器替换同相放大器减去这个直流分量。所幸这里10KHz正弦信号Vpp并不低,不需要放大很多倍,所以使用同相放大器并不会导致其输出饱和。但是接下来就没这么幸运了~

低通滤波器-30KHz

电路图结构没有变,同样也使用了4阶RC低通。

其实最开始我用了2阶MFB有源低通滤波器,可是输出波形非常的诡异...应该是LM324这运放性能太弟弟了,它的参数性能达不到设计要求,导致了输出波形的非线性失真。所以我干脆用无源RC了~

30KHz的正弦信号出来之后,幅度非常小而且带有很高的直流分量...我们需要把它干下去(于是我就弄来一个好东西—— **差分放大器**



其输出Vo和输入Vi2和Vi1的关系是：

$$V_o = (V_{i2} - V_{i1}) * (R_3 / R_1)$$

注意当 $R_4=R_3, R_2=R_1$ 时上式才成立。

在这张图里, $V_o = 10(V_{i2}-V_{i1})$,那我们把从滤波器里爬出来的30KHz正弦信号接到Vi2的位置上,在Vi1上用电位器分压得到一个直流偏置,慢慢拧电位器就能把输出直流偏置调整到合理的区间了~

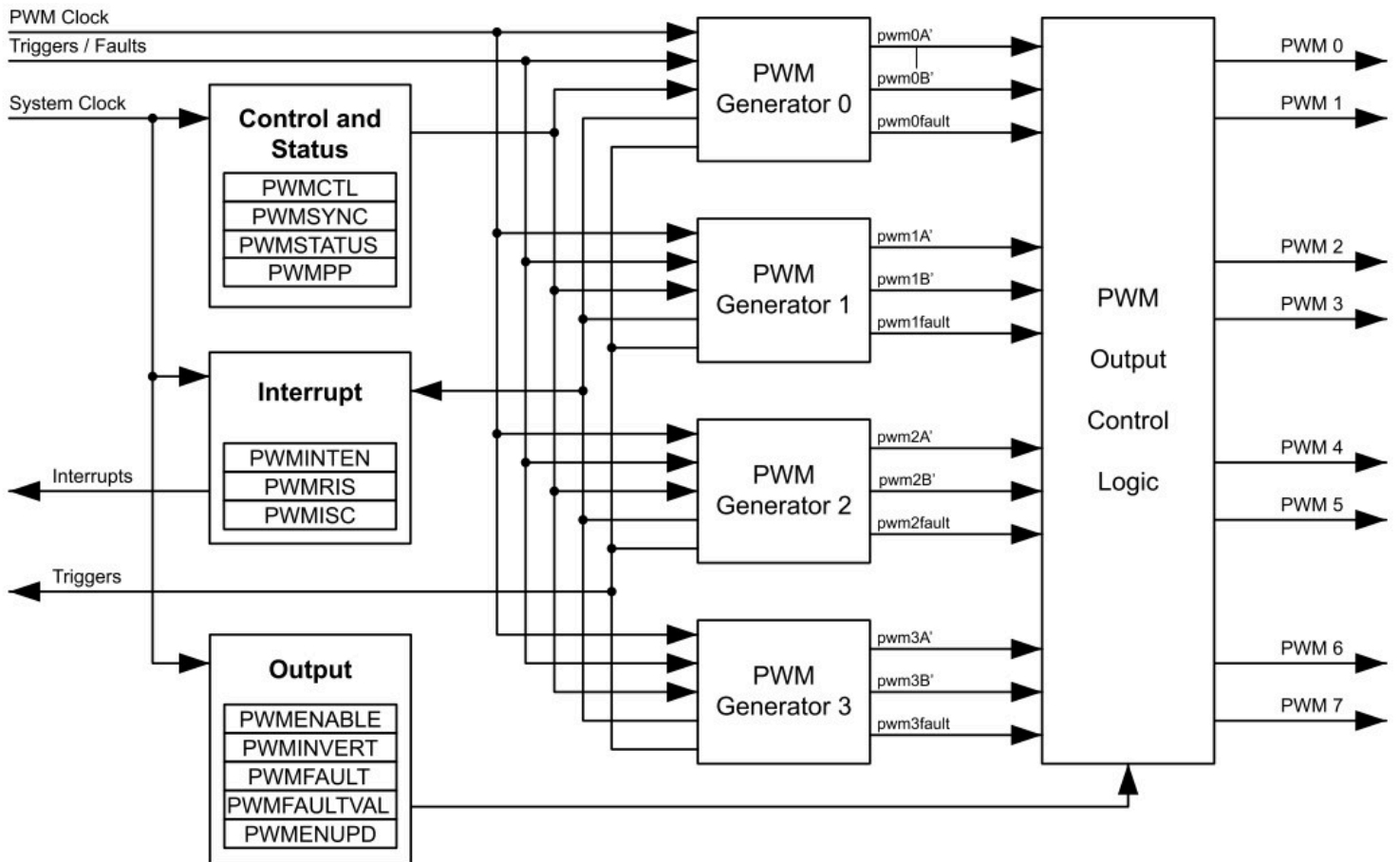
再拧一拧电位器,就能把输出Vpp拧到6V差不多的位置了。

Coding时间到

我就不把所有代码贴上来了,在这里对TM4C123单片机做个简单介绍。

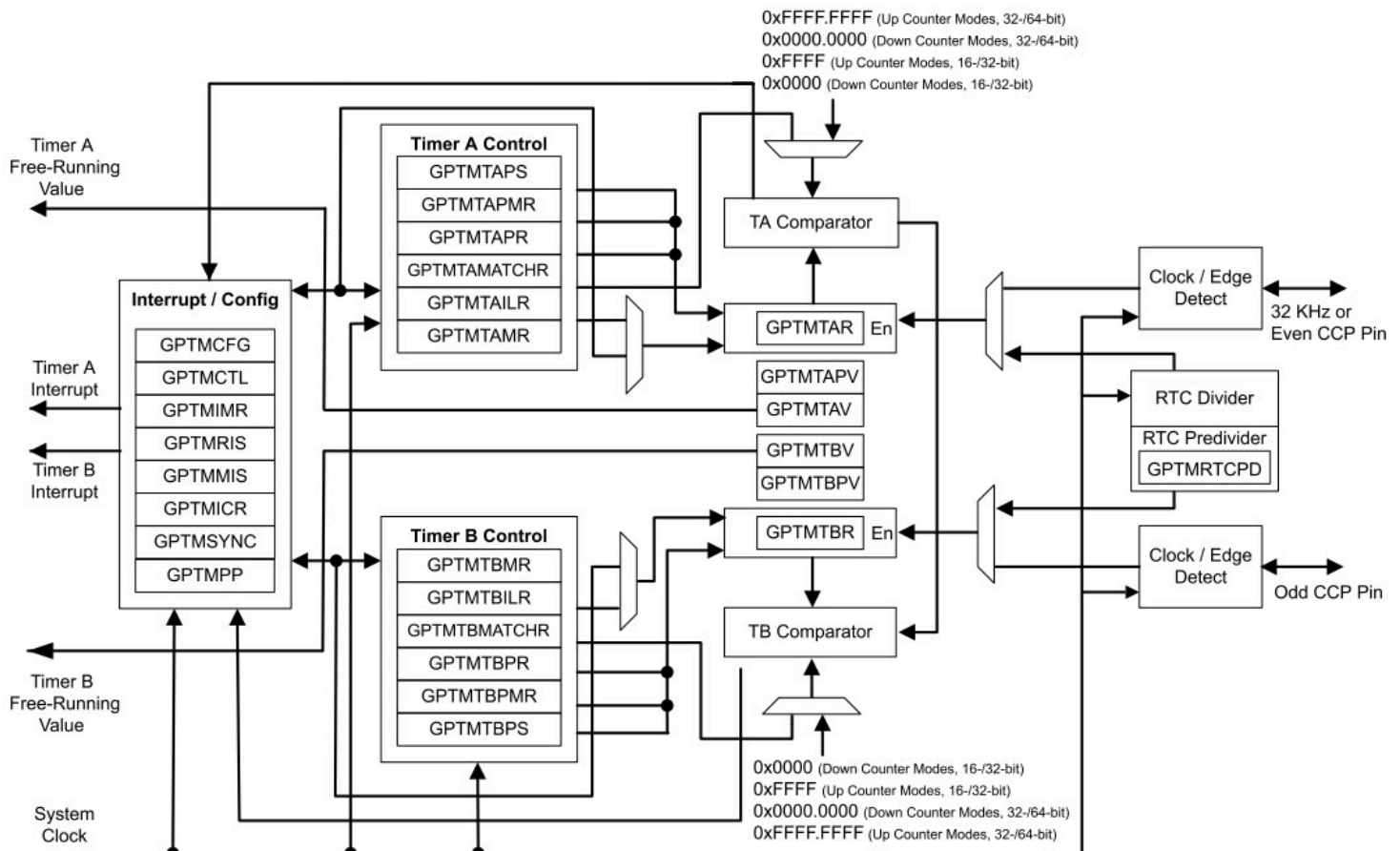
PWM Module

TM4C123内置了4个PWM发生模块,只需要简单地填写参数就能产生PWM信号,这个还是挺好用的。



Timer Module

TM4C123的定时器也真不戳,虽然这个图很复杂,但我们可以不用看...



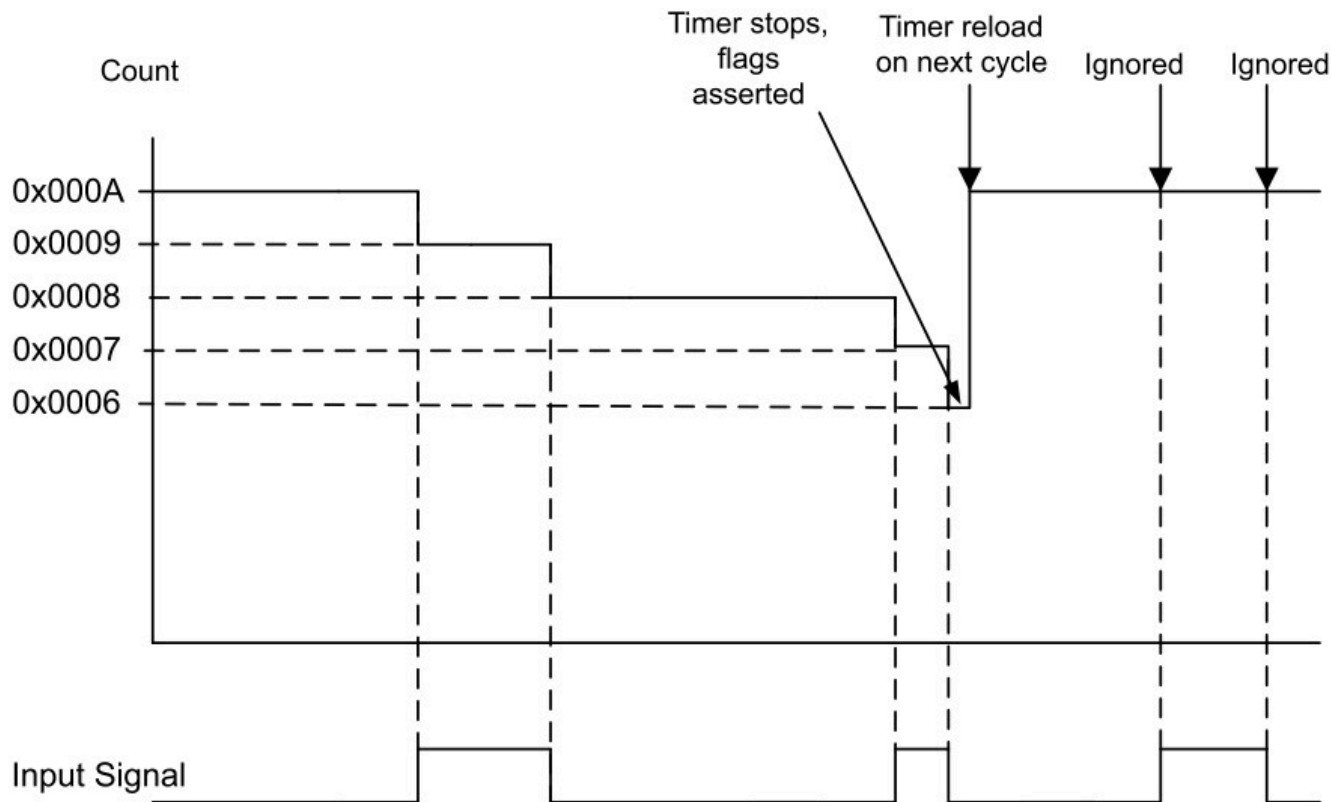
[-]	11. General-Purpose Timers
[-]	11.1. Block Diagram
[-]	11.2. Signal Description
[-]	11.3. Functional Description
[-]	11.3.1. GPTM Reset Conditions
[-]	11.3.2. Timer Modes
	11.3.2.1. One-Shot/Periodic Timer Mode
	11.3.2.2. Real-Time Clock Timer Mode
	11.3.2.3. Input Edge-Count Mode
	11.3.2.4. Input Edge-Time Mode
	11.3.2.5. PWM Mode
	11.3.3. Wait-for-Trigger Mode
	11.3.4. Synchronizing GP Timer Blocks
	11.3.5. DMA Operation
	11.3.6. Accessing Concatenated 16/32-Bit GPTM Register
	11.3.7. Accessing Concatenated 32/64-Bit Wide GPTM Register
[+]	11.4. Initialization and Configuration
	11.5. Register Map
[+]	11.6. Register Descriptions

我们看看定时器的模式,大概有5种,如果用来测频率的话可以用Input-Edge-Count Mode,也就是 输入边沿计数 模式。如果用来测占空比,可以用Input-Edge-Time-Mode,也就是 输入边沿计时 模式。我们主要看前一种测频率的(因为后一种我没调通orz)

首先假定其工作模式为 向下计数,上升沿捕获,自动重装计数值,预装载计数值0xFFFF(16位定时器)

在输入边沿计数模式,定时器不会一直计数,而是等待输入捕获事件的发生,一旦发生,计数器立刻-1(在向上计数模式,是+1)。如果遇上外部复位的指示,那么它暂时忽略输入信号(要抓的信号),并且将计数值重置为默认值或者用户在复位过程中指定的值。

Figure 11-3. Input Edge-Count Mode Example, Counting Down



那么这就是输入边沿计数模式啦~如果我们要测频率,该怎么做呢?

我的思路是这样的,在一段时间内,比如100ms,开启这个定时器(在开启之前先读出计数器现在的值),让他捕获外部脉冲的个数,等待100ms结束之后,读出计数器计数完之后的值,将这2个值相减,再除以0.1(秒),就是所测量信号的频率。

那么怎么做到精准的100ms控制呢? **答案是再开一个定时器(逃)**

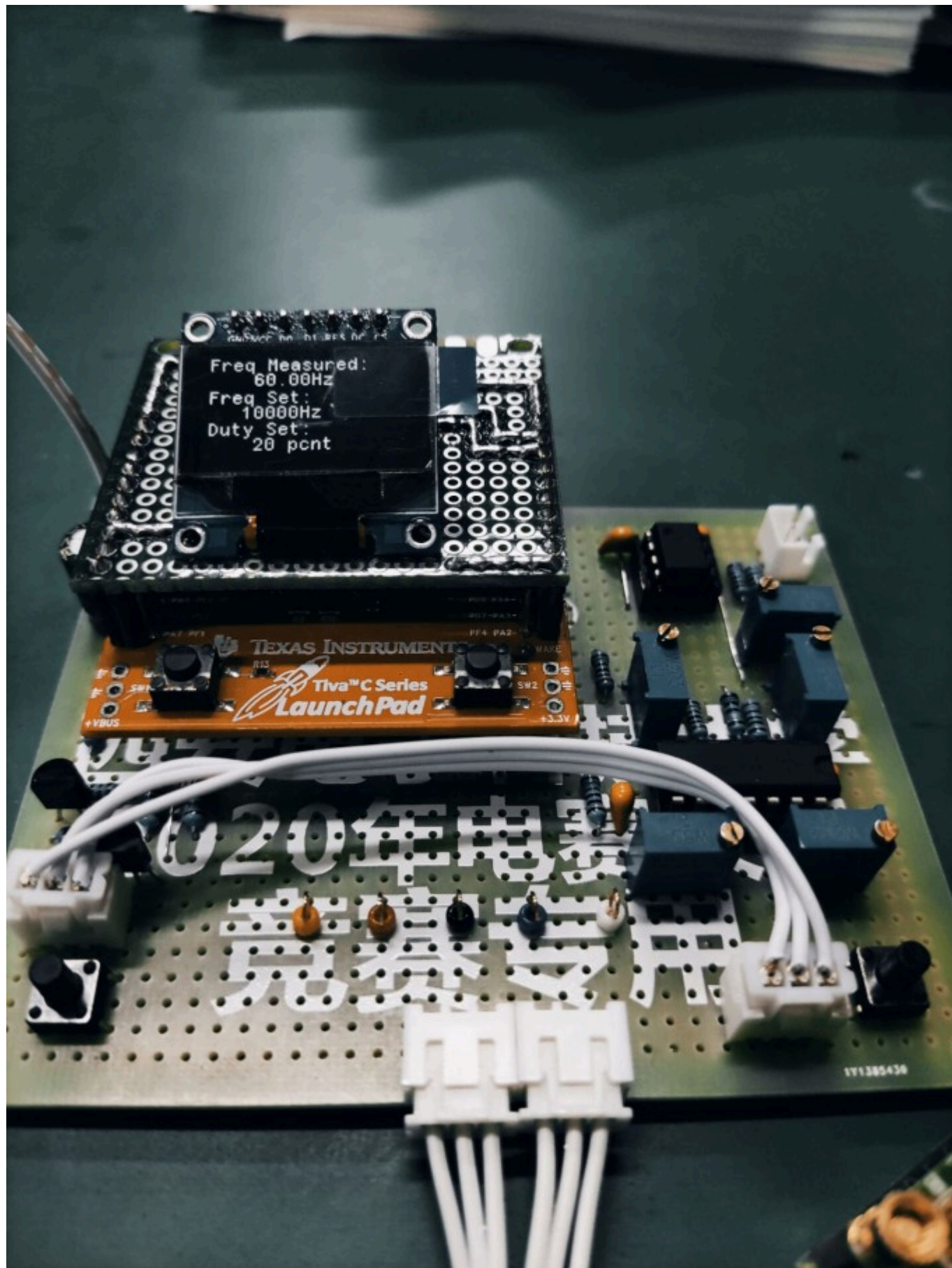
我很认真.jpg

我真的开了2个定时器来测频率,思路就是我上面提到的,每100ms开启/关闭定时器,并且取出测频的那个定时器的计数器的值,相减之后再除以0.1得到频率值。

实测这个思路测频率能测得100Hz-500KHz之间的频率,精度能到10位数,我觉得针滴不戳了~

实物

俺来献个丑吧,做的不是很好看,凑合能用。



代码？

emmmm你真的想看吗orz 如果想的话我整理一下发到github上吧~

(主要是我偷懒了,注释写的不全,怕发出来被打(¬_¬))

下一期

肯定会有下一期的✓
敬请期待~